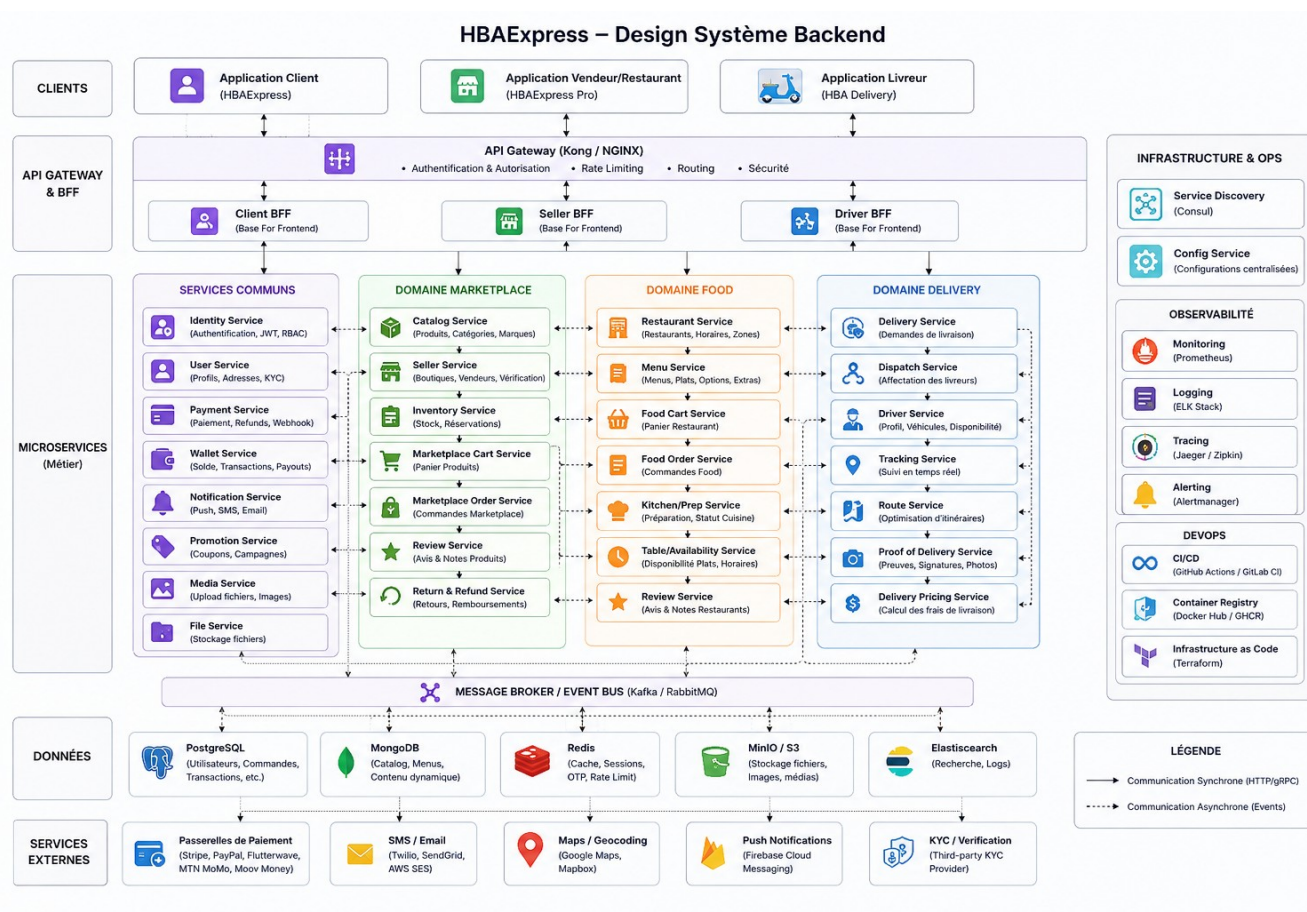


HBAExpress BACKEND

Cahier des charges technique & architecture microservices

Marketplace + Food + Delivery



Architecture de communication : gRPC synchrone + Apache Kafka asynchrone

Version 1.0 - Août 2026

1. Objet du document

Ce cahier des charges définit l'architecture backend de HBAExpress, plateforme multi-activité composée d'un univers Marketplace, d'un univers Food et d'un domaine Delivery partagé. Il sert de référence aux équipes backend, mobile/web, DevOps, QA et sécurité.

Le principe directeur est la séparation des bounded contexts : les paniers et commandes Marketplace ne sont jamais fusionnés avec ceux de Food. Les services transverses (identité, paiement, wallet, notifications, promotions, livraison) sont mutualisés.

2. Principes d'architecture

Décision	Choix	Règle
Clients -> Backend	REST/JSON via API Gateway + BFF	Les applications mobiles/web ne consomment pas directement les microservices.
Synchrone inter-services	gRPC + Protobuf	Timeouts stricts, deadlines, retries contrôlés, mTLS en production.
Asynchrone	Apache Kafka	Événements métier versionnés; Outbox Pattern; consumer idempotent; DLQ.
Données	Database per service	Aucun service ne lit directement la base d'un autre service.
Cohérence	Eventual consistency	Saga orchestration/choreography selon le flux; compensations métier.
Identifiants	UUID v7 conseillé	Traçabilité et génération distribuée.
Montants	BIGINT en unités monétaires entières	XOF sans flottants; amount=8850 signifie 8 850 XOF.
Observabilité	OpenTelemetry	TraceId propagé HTTP -> gRPC -> Kafka.

3. Périmètre des services

#	Service	Domaine	Base principale
1	Identity Service	Commun	PostgreSQL (identity_db) + Redis pour sessions, OTP et révocation de tokens
2	User Service	Commun	PostgreSQL (user_db)
3	Merchant Service	Commun	PostgreSQL (merchant_db) + objet S3/MinIO pour documents KYC
4	Catalog Service	Marketplace	PostgreSQL (catalog_db) + Elasticsearch/OpenSearch pour recherche + S3/MinIO pour médias
5	Inventory Service	Marketplace	PostgreSQL (inventory_db) + Redis pour réservations TTL
6	Marketplace Cart Service	Marketplace	Redis (panier actif) + PostgreSQL (cart_db) pour persistance/audit
7	Marketplace Order Service	Marketplace	PostgreSQL (marketplace_order_db)

8	Restaurant Service	Food	PostgreSQL (restaurant_db) + Redis pour statut temps réel
9	Menu Service	Food	PostgreSQL (menu_db) + Redis cache
10	Food Cart Service	Food	Redis + PostgreSQL (food_cart_db)
11	Food Order Service	Food	PostgreSQL (food_order_db)
12	Payment Service	Commun	PostgreSQL (payment_db) + Kafka + Redis idempotency; données carte jamais stockées
13	Wallet & Settlement Service	Commun	PostgreSQL (wallet_db), modèle double-entry immutable
14	Delivery Service	Delivery	PostgreSQL (delivery_db) + Redis GEO pour disponibilité/positions + TimescaleDB optionnel pour traces GPS
15	Notification Service	Commun	PostgreSQL (notification_db) + Kafka; providers externes FCM/APNs, Twilio/agrégateur SMS, SendGrid/SES
16	Promotion Service	Commun	PostgreSQL (promotion_db) + Redis pour compteurs rapides

4. Topologie logique

Chaîne d'accès : Application Client / HBAExpress Pro / HBA Delivery -> API Gateway -> BFF dédié -> microservices. Les BFF agrègent les réponses gRPC et exposent un contrat JSON stable au frontend.

Exemple d'agrégation BFF

```
{
  "client": "HBAExpress Mobile",
  "http": "GET /api/v1/food/orders/ford_123",
  "gateway": "JWT + rate limit + routing",
  "bff": "Client BFF",
  "grpcCalls": [
    "FoodOrderService.GetFoodOrder",
    "DeliveryService.GetDelivery",
    "UserService.GetUser(driverId)"
  ],
  "response": "JSON agrégé"
}
```

5. Standards REST/JSON externes

Enveloppe succès

```
{
  "success": true,
  "data": {},
  "meta": {
    "requestId": "req_01J...",
    "timestamp": "2026-08-17T20:00:00Z"
  }
}
```

Enveloppe erreur

```
{
  "success": false,
  "error": {
```

```

    "code": "FOOD_ORDER_NOT_FOUND",
    "message": "Commande introuvable",
    "details": []
  },
  "meta": {
    "requestId": "req_01J...",
    "timestamp": "2026-08-17T20:00:00Z"
  }
}

```

HTTP	Usage
200	Lecture/mise à jour réussie
201	Ressource créée
202	Traitement asynchrone accepté
204	Suppression sans body
400	Requête invalide
401	Non authentifié
403	Interdit
404	Introuvable
409	Conflit métier/idempotence
422	Validation métier
429	Rate limit
500	Erreur interne
503	Dépendance indisponible

6. Standards gRPC internes

Les contrats gRPC sont définis dans un repository/package de contrats versionnés. Les messages Protobuf n'exposent jamais les entités ORM. Les appels propagent x-request-id, traceparent, caller-service, principal-id et deadline.

```

syntax = "proto3";
package hba.delivery.v1;

service DeliveryService {
  rpc GetDelivery(GetDeliveryRequest) returns (DeliveryReply);
  rpc CreateDelivery(CreateDeliveryRequest) returns (DeliveryReply);
}

message GetDeliveryRequest { string delivery_id = 1; }
message Money { int64 amount = 1; string currency = 2; }

```

7. Standards Kafka

Convention de topic : hba.<domain>.<aggregate>.<event>.v1. Clé Kafka = aggregateId pour préserver l'ordre par agrégat. Les producteurs écrivent l'événement dans une table Outbox dans la même transaction que la donnée métier; un relay publie ensuite vers Kafka.

Event envelope

```

{
  "eventId": "evt_01J...",
  "eventType": "food.order.accepted",
  "eventVersion": 1,

```

HBAExpress - Cahier des charges Backend

```
"occurredAt": "2026-08-17T20:03:00Z",
"producer": "food-order-service",
"traceId": "00-...",
"aggregate": {
  "type": "FoodOrder",
  "id": "ford_..."
},
"data": {
  "restaurantId": "rest_...",
  "preparationMinutes": 18
}
}
```

Composant	Règle
Partitions	Dimensionnées selon débit; clé par aggregateId
Replication	3 en production si cluster >=3 brokers
acks	all
Idempotence producer	Activée
Consumer	Idempotency store / processed_events
Retries	Backoff + retry topics si nécessaire
DLQ	hba.<service>.dlq.v1
Schema	Protobuf/Avro + Schema Registry recommandé
Retention	Selon nature; événements critiques >= 7 à 30 jours

8. Sécurité

Zone	Exigences
API Gateway	TLS, WAF/rate limit, JWT validation, IP reputation selon besoin
Inter-services	mTLS, service identity, autorisation par service
Secrets	Vault/Kubernetes Secrets chiffrés; jamais dans Git
Paieement	Aucune donnée carte brute; tokens provider uniquement
PII	Chiffrement au repos + politique de rétention
KYC	Object storage privé; URLs signées à durée courte
Audit	Journal immuable des actions admin/finance
OWASP	ASVS/API Security Top 10 comme baseline

9. Architecture des données

Chaque microservice possède son schéma/base. Les jointures cross-service sont interdites. Les vues composées sont produites par BFF, read models ou événements. PostgreSQL est la base transactionnelle par défaut; Redis est utilisé pour cache, sessions, locks courts, paniers et géospatial temps réel; OpenSearch/Elasticsearch sert à la recherche catalogue.

10.1 Identity Service

Authentification, gestion des sessions, rôles, permissions, MFA/OTP, tokens JWT/refresh et identités techniques gRPC.

Responsabilités et limites

Domaine : Commun. Base : PostgreSQL (identity_db) + Redis pour sessions, OTP et révocation de tokens. Le service est propriétaire de ses données et n'autorise aucun accès SQL direct depuis un autre service.

Structure des données

Table / agrégat	Champs principaux
users_auth	id UUID PK, user_id UUID UNIQUE, email CTEXT UNIQUE, phone E.164, password_hash, status, email_verified, phone_verified, created_at, updated_at
roles	id UUID, code UNIQUE, name
permissions	id UUID, code UNIQUE, resource, action
user_roles	user_id, role_id
refresh_tokens	id UUID, user_id, token_hash, expires_at, revoked_at, device_id
mfa_challenges	id UUID, user_id, channel, code_hash, expires_at, attempts

Contrats gRPC exposés

RPC	Usage
ValidateAccessToken(token) -> Principal	ValidateAccessToken
GetUserRoles(userId) -> Roles	GetUserRoles
RevokeUserSessions(userId) -> Empty	RevokeUserSessions

Événements Kafka

Direction	Topics / événements
Publie	identity.user.registered, identity.user.logged_in, identity.token.revoked
Consomme	Aucun

API REST/JSON exposée via BFF/Gateway

POST /api/v1/auth/register

Crée une identité et déclenche la création du profil utilisateur.

Propriété	Valeur
Authentification	Public
Réponse nominale	201 Created
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "email": "client@example.com",
  "phoneNumber": "+2290197000000",
  "password": "*****",
  "firstName": "Awa",
  "lastName": "Dossou"
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "userId": "8c...",
    "email": "client@example.com",
    "status": "PENDING_VERIFICATION"
  },
  "meta": {
    "requestId": "req_..."
  }
}
```

POST /api/v1/auth/login

Authentifie un utilisateur et retourne access/refresh token.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "login": "client@example.com",
  "password": "*****",
  "device": {
    "id": "ios-uuid",
    "name": "iPhone"
  }
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "accessToken": "eyJ...",
    "expiresIn": 900,
    "refreshToken": "rt...",
    "user": {
      "id": "8c...",
      "roles": [
        "CUSTOMER"
      ]
    }
  },
  "meta": {
    "requestId": "req_..."
  }
}
```

POST /api/v1/auth/refresh

Renouvelle la paire de tokens.

Propriété	Valeur
Authentification	Refresh token
Réponse nominale	200 OK

Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon
-----------------	---

Exemple de requête JSON

```
{
  "refreshToken": "rt_..."
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "accessToken": "eyJ...",
    "expiresIn": 900,
    "refreshToken": "rt_new_..."
  }
}
```

POST /api/v1/auth/logout

Révoque le refresh token courant.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "refreshToken": "rt_..."
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "revoked": true
  }
}
```

POST /api/v1/auth/verify-otp

Valide un OTP email/SMS.

Propriété	Valeur
Authentification	Public/challenge token
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "challengeId": "ch_...",
  "code": "123456"
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "verified": true,
    "channel": "SMS"
  }
}
```


Erreurs métier recommandées

Code	HTTP	Description
IDENTITY_SERVICE_NOT_FOUND	404	Ressource principale introuvable.
VALIDATION_ERROR	400	Payload invalide ou champ obligatoire absent.
BUSINESS_RULE_VIOLATION	422	Règle métier non satisfaite.
CONFLICT	409	État incompatible, version concurrente ou idempotency conflict.
DEPENDENCY_UNAVAILABLE	503	Dépendance gRPC/Kafka/provider indisponible.

10.2 User Service

Profils clients, vendeurs, livreurs, adresses, préférences et pièces de profil.

Responsabilités et limites

Domaine : Commun. Base : PostgreSQL (user_db). Le service est propriétaire de ses données et n'autorise aucun accès SQL direct depuis un autre service.

Structure des données

Table / agrégat	Champs principaux
users	id UUID PK, first_name, last_name, display_name, phone, email, avatar_url, locale, status, created_at
addresses	id UUID, user_id, label, street, city, country_code, lat, lng, is_default
preferences	user_id PK, language, currency, push_enabled, marketing_opt_in
devices	id UUID, user_id, platform, push_token, last_seen_at

Contrats gRPC exposés

RPC	Usage
GetUser(userId)->User	GetUser
GetAddress(addressId)->Address	GetAddress
ListUserAddresses(userId)->Addresses	ListUserAddresses

Événements Kafka

Direction	Topics / événements
Publie	user.profile.updated, user.address.created, user.device.registered
Consomme	identity.user.registered

API REST/JSON exposée via BFF/Gateway

GET /api/v1/users/me

Retourne le profil connecté.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "id": "u...",
    "firstName": "Awa",
    "lastName": "Dossou",
    "email": "client@example.com",
```

```

    "phoneNumber": "+229...",
    "locale": "fr-BJ",
    "currency": "XOF",
    "avatarUrl": null
  }
}

```

PATCH /api/v1/users/me

Met à jour les champs éditables.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```

{
  "displayName": "Awa D.",
  "locale": "fr-BJ"
}

```

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "id": "u...",
    "displayName": "Awa D.",
    "locale": "fr-BJ",
    "updatedAt": "2026-08-17T19:00:00Z"
  }
}

```

POST /api/v1/users/me/addresses

Ajoute une adresse.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	201 Created
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```

{
  "label": "Maison",
  "street": "Rue 123",
  "city": "Cotonou",
  "countryCode": "BJ",
  "location": {
    "lat": 6.37,
    "lng": 2.39
  },
  "isDefault": true
}

```

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "id": "addr...",
    "label": "Maison",
    "city": "Cotonou",
    "isDefault": true
  }
}

```

GET /api/v1/users/me/addresses

Liste les adresses.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```
{
  "success": true,
  "data": [
    {
      "id": "addr_...",
      "label": "Maison",
      "street": "Rue 123",
      "city": "Cotonou",
      "location": {
        "lat": 6.37,
        "lng": 2.39
      },
      "isDefault": true
    }
  ]
}
```

Erreurs métier recommandées

Code	HTTP	Description
USER_SERVICE_NOT_FOUND	404	Ressource principale introuvable.
VALIDATION_ERROR	400	Payload invalide ou champ obligatoire absent.
BUSINESS_RULE_VIOLATION	422	Règle métier non satisfaite.
CONFLICT	409	État incompatible, version concurrente ou idempotency conflict.
DEPENDENCY_UNAVAILABLE	503	Dépendance gRPC/Kafka/provider indisponible.

10.3 Merchant Service

Onboarding vendeurs/restaurants, boutiques, établissements, KYC, membres d'équipe, rôles métier et paramètres commerciaux.

Responsabilités et limites

Domaine : Commun. Base : PostgreSQL (merchant_db) + objet S3/MinIO pour documents KYC. Le service est propriétaire de ses données et n'autorise aucun accès SQL direct depuis un autre service.

Structure des données

Table / agrégat	Champs principaux
merchants	id UUID, owner_user_id, legal_name, trade_name, registration_number, tax_id, kyc_status, status
outlets	id UUID, merchant_id, type SHOP RESTAURANT, name, address_id, lat, lng, status
merchant_members	merchant_id, user_id, role, permissions_json
kyc_documents	id, merchant_id, type, object_key, status, reviewed_at

Contrats gRPC exposés

RPC	Usage
GetMerchant(merchantId)	GetMerchant
GetOutlet(outletId)	GetOutlet
CheckMerchantCapability(merchantId, capability)	CheckMerchantCapability

Événements Kafka

Direction	Topics / événements
Publie	merchant.created, merchant.kyc.submitted, merchant.kyc.approved, outlet.status.changed
Consomme	identity.user.registered

API REST/JSON exposée via BFF/Gateway

POST /api/v1/merchants

Crée le compte professionnel.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	201 Created
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "legalName": "HBA Foods SARL",
  "tradeName": "Chez Awa",
}
```

```

    "registrationNumber": "RCCM-...",
    "outletType": "RESTAURANT"
  }

```

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "merchantId": "m_...",
    "kycStatus": "DRAFT",
    "status": "PENDING"
  }
}

```

POST /api/v1/merchants/{id}/outlets

Crée une boutique ou un restaurant.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	201 Created
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```

{
  "type": "SHOP",
  "name": "HBA Fashion",
  "addressId": "addr_...",
  "location": {
    "lat": 6.37,
    "lng": 2.39
  }
}

```

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "id": "out_...",
    "type": "SHOP",
    "name": "HBA Fashion",
    "status": "DRAFT"
  }
}

```

GET /api/v1/merchants/{id}

Retourne le marchand et ses établissements.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "id": "m_...",
    "tradeName": "Chez Awa",
    "kycStatus": "APPROVED",
    "outlets": [
      {
        "id": "out_1",
        "type": "SHOP",

```

```

    "status": "ACTIVE"
  },
  {
    "id": "out_2",
    "type": "RESTAURANT",
    "status": "ACTIVE"
  }
]
}
}

```

POST /api/v1/merchants/{id}/kyc/submit

Soumet le dossier KYC.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```

{
  "documentIds": [
    "doc_1",
    "doc_2"
  ]
}

```

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "merchantId": "m_...",
    "kycStatus": "UNDER_REVIEW"
  }
}

```

Erreurs métier recommandées

Code	HTTP	Description
MERCHANT_SERVICE_NOT_FOUND	404	Ressource principale introuvable.
VALIDATION_ERROR	400	Payload invalide ou champ obligatoire absent.
BUSINESS_RULE_VIOLATION	422	Règle métier non satisfaite.
CONFLICT	409	État incompatible, version concurrente ou idempotency conflict.
DEPENDENCY_UNAVAILABLE	503	Dépendance gRPC/Kafka/provider indisponible.

10.4 Catalog Service

Catalogue des articles non alimentaires : produits, variantes, catégories, marques, attributs, images et recherche.

Responsabilités et limites

Domaine : Marketplace. Base : PostgreSQL (catalog_db) + Elasticsearch/OpenSearch pour recherche + S3/MinIO pour médias. Le service est propriétaire de ses données et n'autorise aucun accès SQL direct depuis un autre service.

Structure des données

Table / agrégat	Champs principaux
products	id UUID, merchant_id, outlet_id, category_id, brand_id, name, slug, description, status, created_at
variants	id UUID, product_id, sku, barcode, price_xof BIGINT, compare_at_price_xof, attributes_json, status
categories	id UUID, parent_id, name, slug, level, status
product_media	id, product_id, url, type, display_order, is_main

Contrats gRPC exposés

RPC	Usage
GetProduct(productId)	GetProduct
GetVariant(variantId)	GetVariant
SearchProducts(query, filters)	SearchProducts

Événements Kafka

Direction	Topics / événements
Publie	catalog.product.created, catalog.product.updated, catalog.product.published
Consomme	inventory.stock.changed

API REST/JSON exposée via BFF/Gateway

POST /api/v1/marketplace/products

Crée un produit et ses variantes.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	201 Created
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "outletId": "out_shop",
  "name": "Sneakers Elite",
  "categoryId": "cat_shoes",
  "description": "...",

```



```

"variants": [
  {
    "sku": "SNK-42-BLK",
    "price": 25000,
    "attributes": {
      "size": "42",
      "color": "BLACK"
    }
  }
]
}

```

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "id": "prod...",
    "status": "DRAFT",
    "variants": [
      {
        "id": "var...",
        "sku": "SNK-42-BLK",
        "price": 25000,
        "currency": "XOF"
      }
    ]
  }
}

```

GET /api/v1/marketplace/products/{id}

Détail produit.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "id": "prod...",
    "name": "Sneakers Elite",
    "priceRange": {
      "min": 25000,
      "max": 28000,
      "currency": "XOF"
    },
    "images": [
      {
        "url": "https://cdn/...",
        "isMain": true
      }
    ],
    "variants": [
      {
        "id": "var...",
        "sku": "SNK-42-BLK",
        "attributes": {
          "size": "42",
          "color": "BLACK"
        },
        "available": true
      }
    ]
  }
}

```

Recherche/pagination produits.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```
{
  "success": true,
  "data": [
    {
      "id": "prod_...",
      "name": "Sneakers Elite",
      "mainImage": "https://cdn/...",
      "price": 25000,
      "currency": "XOF",
      "rating": 4.7
    }
  ],
  "meta": {
    "page": 1,
    "pageSize": 20,
    "total": 134,
    "hasNext": true
  }
}
```

PATCH /api/v1/marketplace/products/{id}

Met à jour un produit.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "name": "Sneakers Elite 2026",
  "status": "ACTIVE"
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "id": "prod_...",
    "name": "Sneakers Elite 2026",
    "status": "ACTIVE"
  }
}
```

Erreurs métier recommandées

Code	HTTP	Description
CATALOG_SERVICE_NOT_FOUND	404	Ressource principale introuvable.
VALIDATION_ERROR	400	Payload invalide ou champ obligatoire absent.

HBAExpress - Cahier des charges Backend

BUSINESS_RULE_VIOLATION	422	Règle métier non satisfaite.
CONFLICT	409	État incompatible, version concurrente ou idempotency conflict.
DEPENDENCY_UNAVAILABLE	503	Dépendance gRPC/Kafka/provider indisponible.

10.5 Inventory Service

Stock des variantes, réservations temporaires, mouvements, seuils d'alerte et contrôle de disponibilité.

Responsabilités et limites

Domaine : Marketplace. Base : PostgreSQL (inventory_db) + Redis pour réservations TTL. Le service est propriétaire de ses données et n'autorise aucun accès SQL direct depuis un autre service.

Structure des données

Table / agrégat	Champs principaux
stocks	variant_id UUID PK, outlet_id, on_hand INT, reserved INT, available INT generated/logical, reorder_level
stock_movements	id UUID, variant_id, type, quantity, reference_type, reference_id, created_at
reservations	id UUID, cart_id, variant_id, quantity, expires_at, status

Contrats gRPC exposés

RPC	Usage
CheckAvailability(items)->Availability	CheckAvailability
ReserveStock(cartId, items)->Reservation	ReserveStock
CommitReservation(orderId, reservationId)	CommitReservation
ReleaseReservation(reservationId)	ReleaseReservation

Événements Kafka

Direction	Topics / événements
Publie	inventory.stock.changed, inventory.reservation.expired, inventory.out_of_stock
Consomme	marketplace.order.cancelled, marketplace.order.paid

API REST/JSON exposée via BFF/Gateway

GET /api/v1/inventory/variants/{variantId}

Etat de stock vendeur.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "variantId": "var_...",
    "onHand": 40,
  }
}
```

```

    "reserved": 5,
    "available": 35,
    "reorderLevel": 10
  }
}

```

POST /api/v1/inventory/adjustments

Ajuste le stock avec audit.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	201 Created
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```

{
  "variantId": "var_...",
  "quantityDelta": 20,
  "reason": "PURCHASE_RECEIPT",
  "reference": "P0-2026-001"
}

```

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "movementId": "mov_...",
    "onHand": 60,
    "available": 55
  }
}

```

POST /api/v1/inventory/check

Vérifie la disponibilité avant checkout.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```

{
  "items": [
    {
      "variantId": "var_...",
      "quantity": 2
    }
  ]
}

```

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "available": true,
    "items": [
      {
        "variantId": "var_...",
        "requested": 2,
        "availableQuantity": 35,
        "isAvailable": true
      }
    ]
  }
}

```

```
}  
}
```

Erreurs métier recommandées

Code	HTTP	Description
INVENTORY_SERVICE_NOT_FOUND	404	Ressource principale introuvable.
VALIDATION_ERROR	400	Payload invalide ou champ obligatoire absent.
BUSINESS_RULE_VIOLATION	422	Règle métier non satisfaite.
CONFLICT	409	État incompatible, version concurrente ou idempotency conflict.
DEPENDENCY_UNAVAILABLE	503	Dépendance gRPC/Kafka/provider indisponible.

10.6 Marketplace Cart Service

Panier des articles Marketplace, calcul du sous-total, regroupement par vendeur, coupon et snapshot de prix.

Responsabilités et limites

Domaine : Marketplace. Base : Redis (panier actif) + PostgreSQL (cart_db) pour persistance/audit. Le service est propriétaire de ses données et n'autorise aucun accès SQL direct depuis un autre service.

Structure des données

Table / agrégat	Champs principaux
carts	id UUID, user_id, status, currency, subtotal, discount_total, expires_at
cart_items	id UUID, cart_id, variant_id, merchant_id, quantity, unit_price_snapshot, metadata_json

Contrats gRPC exposés

RPC	Usage
GetCart(cartId)	GetCart
ValidateCart(cartId)	ValidateCart
ClearCart(cartId)	ClearCart

Événements Kafka

Direction	Topics / événements
Publie	marketplace.cart.updated, marketplace.cart.checked_out
Consomme	catalog.product.updated, inventory.stock.changed

API REST/JSON exposée via BFF/Gateway

GET /api/v1/marketplace/cart

Retourne le panier courant.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "id": "cart_...",
    "currency": "XOF",
    "items": [
      {
        "id": "ci_...",
        "productId": "prod_...",
        "variantId": "var_...",
        "name": "Sneakers Elite",

```

```

      "quantity": 2,
      "unitPrice": 25000,
      "lineTotal": 50000
    }
  ],
  "summary": {
    "subtotal": 50000,
    "discount": 0,
    "estimatedDelivery": 2000,
    "total": 52000
  }
}

```

POST /api/v1/marketplace/cart/items

Ajoute une variante.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	201 Created
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```

{
  "variantId": "var_...",
  "quantity": 2
}

```

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "cartId": "cart_...",
    "item": {
      "id": "ci_...",
      "variantId": "var_...",
      "quantity": 2
    },
    "summary": {
      "subtotal": 50000,
      "total": 50000
    }
  }
}

```

PATCH /api/v1/marketplace/cart/items/{itemId}

Modifie la quantité.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```

{
  "quantity": 3
}

```

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "itemId": "ci_...",

```



```

    "quantity": 3,
    "lineTotal": 75000,
    "summary": {
      "subtotal": 75000,
      "total": 75000
    }
  }
}

```

DELETE /api/v1/marketplace/cart/items/{itemId}

Retire un article.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "removed": true,
    "summary": {
      "subtotal": 0,
      "total": 0
    }
  }
}

```

Erreurs métier recommandées

Code	HTTP	Description
MARKETPLACE_CART_SERVICE_NOT_FOUND	404	Ressource principale introuvable.
VALIDATION_ERROR	400	Payload invalide ou champ obligatoire absent.
BUSINESS_RULE_VIOLATION	422	Règle métier non satisfaite.
CONFLICT	409	État incompatible, version concurrente ou idempotency conflict.
DEPENDENCY_UNAVAILABLE	503	Dépendance gRPC/Kafka/provider indisponible.

10.7 Marketplace Order Service

Checkout et cycle de vie des commandes Marketplace, séparation par vendeur, historique et orchestration métier.

Responsabilités et limites

Domaine : Marketplace. Base : PostgreSQL (marketplace_order_db). Le service est propriétaire de ses données et n'autorise aucun accès SQL direct depuis un autre service.

Structure des données

Table / agrégat	Champs principaux
orders	id UUID, user_id, address_id, status, payment_status, subtotal, shipping_total, discount_total, total, currency, created_at
order_sellers	id UUID, order_id, merchant_id, outlet_id, status, subtotal
order_items	id UUID, order_seller_id, product_id, variant_id, sku, name_snapshot, unit_price, quantity, line_total
order_status_history	id, order_id, from_status, to_status, actor_id, created_at

Contrats gRPC exposés

RPC	Usage
GetMarketplaceOrder(orderId)	GetMarketplaceOrder
UpdatePaymentStatus(orderId,status)	UpdatePaymentStatus
MarkReadyForPickup(orderId)	MarkReadyForPickup
CancelOrder(orderId,reason)	CancelOrder

Événements Kafka

Direction	Topics / événements
Publie	marketplace.order.created, marketplace.order.paid, marketplace.order.ready_for_pickup, marketplace.order.cancelled, marketplace.order.delivered
Consomme	payment.succeeded, payment.failed, delivery.delivered

API REST/JSON exposée via BFF/Gateway

POST /api/v1/marketplace/orders/checkout

Crée une commande depuis le panier.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	201 Created
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "cartId": "cart_...",
  "deliveryAddressId": "addr_...",
  "paymentMethod": "MTN_MOMO",
  "couponCode": null
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "orderId": "ord_m_...",
    "status": "PENDING_PAYMENT",
    "payment": {
      "paymentIntentId": "pay_...",
      "amount": 52000,
      "currency": "XOF"
    },
    "summary": {
      "subtotal": 50000,
      "delivery": 2000,
      "total": 52000
    }
  }
}
```

GET /api/v1/marketplace/orders/{id}

Détail d'une commande.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "id": "ord_m_...",
    "status": "PREPARING",
    "paymentStatus": "PAID",
    "items": [
      {
        "name": "Sneakers Elite",
        "sku": "SNK-42-BLK",
        "quantity": 2,
        "unitPrice": 25000
      }
    ],
    "delivery": {
      "deliveryId": "del_...",
      "status": "ASSIGNED"
    },
    "total": 52000,
    "currency": "XOF"
  }
}
```

GET /api/v1/marketplace/orders

Historique paginé.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK

Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon
-----------------	---

Exemple de réponse JSON

```
{
  "success": true,
  "data": [
    {
      "id": "ord_m...",
      "status": "DELIVERED",
      "total": 52000,
      "currency": "XOF",
      "createdAt": "2026-08-17T18:00:00Z"
    }
  ],
  "meta": {
    "page": 1,
    "pageSize": 20,
    "total": 8
  }
}
```

POST /api/v1/marketplace/orders/{id}/cancel

Annule si les règles le permettent.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "reasonCode": "CUSTOMER_CHANGED_MIND",
  "comment": "Erreur de taille"
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "orderId": "ord_m...",
    "status": "CANCELLED",
    "refund": {
      "status": "PENDING",
      "amount": 52000
    }
  }
}
```

Erreurs métier recommandées

Code	HTTP	Description
MARKETPLACE_ORDER_SERVICE_NOT_FOUND	404	Ressource principale introuvable.
VALIDATION_ERROR	400	Payload invalide ou champ obligatoire absent.
BUSINESS_RULE_VIOLATION	422	Règle métier non satisfaite.
CONFLICT	409	État incompatible, version concurrente ou idempotency conflict.
DEPENDENCY_UNAVAILABLE	503	Dépendance gRPC/Kafka/provider indisponible.

10.8 Restaurant Service

Référentiel restaurants : établissement, horaires, zones, capacité, disponibilité et état ouvert/fermé.

Responsabilités et limites

Domaine : Food. Base : PostgreSQL (restaurant_db) + Redis pour statut temps réel. Le service est propriétaire de ses données et n'autorise aucun accès SQL direct depuis un autre service.

Structure des données

Table / agrégat	Champs principaux
restaurants	id UUID, merchant_id, outlet_id, name, cuisine_types[], min_order, avg_prep_minutes, status
opening_hours	id, restaurant_id, weekday, open_time, close_time
service_zones	id, restaurant_id, polygon_geojson, fee_policy_id
restaurant_runtime	restaurant_id, is_open, pause_until, capacity_state

Contrats gRPC exposés

RPC	Usage
GetRestaurant(id)	GetRestaurant
IsRestaurantOpen(id,at)	IsRestaurantOpen
GetRestaurantRuntime(id)	GetRestaurantRuntime

Événements Kafka

Direction	Topics / événements
Publie	restaurant.opened, restaurant.closed, restaurant.paused
Consomme	merchant.kyc.approved

API REST/JSON exposée via BFF/Gateway

GET /api/v1/food/restaurants

Liste restaurants disponibles selon position.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```
{
  "success": true,
  "data": [
    {
      "id": "rest_...",
      "name": "Chez Awa",
      "cuisines": [
```

```

    "AFRICAIN"
  ],
  "rating": 4.8,
  "isOpen": true,
  "estimatedDeliveryMinutes": 32,
  "deliveryFee": 1000
}
],
"meta": {
  "page": 1,
  "pageSize": 20,
  "total": 45
}
}

```

GET /api/v1/food/restaurants/{id}

Détail restaurant.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "id": "rest_...",
    "name": "Chez Awa",
    "isOpen": true,
    "openingHours": [
      {
        "day": "MONDAY",
        "open": "10:00",
        "close": "23:00"
      }
    ],
    "averagePrepMinutes": 18,
    "minimumOrder": 2500,
    "currency": "XOF"
  }
}

```

PATCH /api/v1/food/restaurants/{id}/runtime

Ouvre, ferme ou met en pause.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```

{
  "state": "PAUSED",
  "pauseMinutes": 30
}

```

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "restaurantId": "rest_...",
    "isOpen": false,
  }
}

```

HBAExpress - Cahier des charges Backend

```
"pauseUntil": "2026-08-17T20:30:00Z"
}
```

Erreurs métier recommandées

Code	HTTP	Description
RESTAURANT_SERVICE_NOT_FOUND	404	Ressource principale introuvable.
VALIDATION_ERROR	400	Payload invalide ou champ obligatoire absent.
BUSINESS_RULE_VIOLATION	422	Règle métier non satisfaite.
CONFLICT	409	État incompatible, version concurrente ou idempotency conflict.
DEPENDENCY_UNAVAILABLE	503	Dépendance gRPC/Kafka/provider indisponible.

10.9 Menu Service

Menus, catégories, plats, options, extras, prix, indisponibilités et images.

Responsabilités et limites

Domaine : Food. Base : PostgreSQL (menu_db) + Redis cache. Le service est propriétaire de ses données et n'autorise aucun accès SQL direct depuis un autre service.

Structure des données

Table / agrégat	Champs principaux
menus	id UUID, restaurant_id, name, status
menu_categories	id, menu_id, name, display_order
menu_items	id UUID, category_id, name, description, base_price, status, image_url, prep_minutes_override
option_groups	id, menu_item_id, name, min_select, max_select, required
options	id, group_id, name, price_delta, status

Contrats gRPC exposés

RPC	Usage
GetMenuItem(itemId)	GetMenuItem
ValidateMenuSelection(itemId, optionIds)	ValidateMenuSelection
GetRestaurantMenu(restaurantId)	GetRestaurantMenu

Événements Kafka

Direction	Topics / événements
Publie	menu.item.updated, menu.item.unavailable, menu.published
Consomme	Aucun

API REST/JSON exposée via BFF/Gateway

GET /api/v1/food/restaurants/{id}/menu

Retourne le menu actif.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "restaurantId": "rest_...",

```



```

"categories": [
  {
    "id": "mc_1",
    "name": "Plats",
    "items": [
      {
        "id": "dish_...",
        "name": "Poulet DG",
        "price": 3500,
        "currency": "XOF",
        "available": true,
        "optionGroups": [
          {
            "id": "og_1",
            "name": "Accompagnement",
            "required": true,
            "min": 1,
            "max": 1,
            "options": [
              {
                "id": "opt_rice",
                "name": "Riz",
                "priceDelta": 0
              }
            ]
          }
        ]
      }
    ]
  }
]
}

```

POST /api/v1/food/menu/items

Crée un plat.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	201 Created
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```

{
  "categoryId": "mc_1",
  "name": "Poulet DG",
  "description": "...",
  "basePrice": 3500,
  "optionGroups": [
    {
      "name": "Accompagnement",
      "required": true,
      "minSelect": 1,
      "maxSelect": 1,
      "options": [
        {
          "name": "Riz",
          "priceDelta": 0
        }
      ]
    }
  ]
}

```

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "id": "dish_...",
  }
}

```

```

    "name": "Poulet DG",
    "basePrice": 3500,
    "status": "ACTIVE"
  }
}

```

PATCH /api/v1/food/menu/items/{id}/availability

Active/désactive temporairement un plat.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```

{
  "available": false,
  "reason": "SOLD_OUT"
}

```

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "id": "dish...",
    "available": false,
    "reason": "SOLD_OUT"
  }
}

```

Erreurs métier recommandées

Code	HTTP	Description
MENU_SERVICE_NOT_FOUND	404	Ressource principale introuvable.
VALIDATION_ERROR	400	Payload invalide ou champ obligatoire absent.
BUSINESS_RULE_VIOLATION	422	Règle métier non satisfaite.
CONFLICT	409	État incompatible, version concurrente ou idempotency conflict.
DEPENDENCY_UNAVAILABLE	503	Dépendance gRPC/Kafka/provider indisponible.

10.10 Food Cart Service

Panier mono-restaurant avec options, extras, notes, validation restaurant et estimation de frais.

Responsabilités et limites

Domaine : Food. Base : Redis + PostgreSQL (food_cart_db). Le service est propriétaire de ses données et n'autorise aucun accès SQL direct depuis un autre service.

Structure des données

Table / agrégat	Champs principaux
food_carts	id UUID, user_id, restaurant_id, status, currency, subtotal, discount_total, expires_at
food_cart_items	id, cart_id, menu_item_id, quantity, base_price_snapshot, options_json, note, line_total

Contrats gRPC exposés

RPC	Usage
GetFoodCart(cartId)	GetFoodCart
ValidateFoodCart(cartId)	ValidateFoodCart
ClearFoodCart(cartId)	ClearFoodCart

Événements Kafka

Direction	Topics / événements
Publie	food.cart.updated, food.cart.checked_out
Consomme	menu.item.unavailable, restaurant.closed

API REST/JSON exposée via BFF/Gateway

GET /api/v1/food/cart

Retourne le panier Food.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "id": "fcart_...",
    "restaurant": {
      "id": "rest_...",
      "name": "Chez Awa"
    },
    "items": [
      {
```

```

    "id": "fci_...",
    "menuItem": "dish_...",
    "name": "Poulet DG",
    "quantity": 2,
    "options": [
      {
        "name": "Riz",
        "priceDelta": 0
      }
    ],
    "unitPrice": 3500,
    "lineTotal": 7000
  }
],
"summary": {
  "subtotal": 7000,
  "delivery": 1000,
  "serviceFee": 350,
  "total": 8350,
  "currency": "XOF"
}
}
}

```

POST /api/v1/food/cart/items

Ajoute un plat. Si restaurant différent, le BFF exige confirmation avant remplacement du panier.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	201 Created
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```

{
  "menuItem": "dish_...",
  "quantity": 2,
  "optionIds": [
    "opt_rice"
  ],
  "note": "Sans piment"
}

```

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "cartId": "fcart_...",
    "itemId": "fci_...",
    "quantity": 2,
    "lineTotal": 7000,
    "summary": {
      "subtotal": 7000,
      "total": 7000
    }
  }
}

```

DELETE /api/v1/food/cart

Vide le panier Food.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK

Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon
-----------------	---

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "cleared": true
  }
}
```

Erreurs métier recommandées

Code	HTTP	Description
FOOD_CART_SERVICE_NOT_FOUND	404	Ressource principale introuvable.
VALIDATION_ERROR	400	Payload invalide ou champ obligatoire absent.
BUSINESS_RULE_VIOLATION	422	Règle métier non satisfaite.
CONFLICT	409	État incompatible, version concurrente ou idempotency conflict.
DEPENDENCY_UNAVAILABLE	503	Dépendance gRPC/Kafka/provider indisponible.

10.11 Food Order Service

Commandes restaurant temps réel : paiement, acceptation/refus, préparation, prêt, pickup et livraison.

Responsabilités et limites

Domaine : Food. Base : PostgreSQL (food_order_db). Le service est propriétaire de ses données et n'autorise aucun accès SQL direct depuis un autre service.

Structure des données

Table / agrégat	Champs principaux
food_orders	id UUID, user_id, restaurant_id, address_id, status, payment_status, subtotal, delivery_fee, service_fee, discount_total, total, requested_at
food_order_items	id, order_id, menu_item_id, name_snapshot, quantity, unit_price, options_json, note
food_status_history	id, order_id, from_status, to_status, actor_id, reason, created_at

Contrats gRPC exposés

RPC	Usage
GetFoodOrder(id)	GetFoodOrder
AcceptOrder(id,prepMinutes)	AcceptOrder
RejectOrder(id,reason)	RejectOrder
MarkReady(id)	MarkReady
UpdatePaymentStatus(id,status)	UpdatePaymentStatus

Événements Kafka

Direction	Topics / événements
Publie	food.order.created, food.order.paid, food.order.accepted, food.order.rejected, food.order.ready, food.order.cancelled, food.order.delivered
Consomme	payment.succeeded, payment.failed, delivery.picked_up, delivery.delivered

API REST/JSON exposée via BFF/Gateway

POST /api/v1/food/orders/checkout

Crée une commande Food.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	201 Created
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "cartId": "fcart_...",
  "deliveryAddressId": "addr_...",
  "paymentMethod": "MTN_MOMO",
  "tip": 500
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "orderId": "ford_...",
    "status": "PENDING_PAYMENT",
    "restaurantConfirmationRequired": true,
    "payment": {
      "paymentIntentId": "pay_...",
      "amount": 8850,
      "currency": "XOF"
    },
    "estimatedDeliveryMinutes": 35
  }
}
```

POST /api/v1/food/orders/{id}/accept

Le restaurant accepte et annonce le temps de préparation.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "preparationMinutes": 18
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "orderId": "ford_...",
    "status": "PREPARING",
    "preparationMinutes": 18,
    "estimatedReadyAt": "2026-08-17T20:18:00Z"
  }
}
```

POST /api/v1/food/orders/{id}/reject

Refus restaurant.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "reasonCode": "KITCHEN_OVERLOADED",
  "comment": "Capacité atteinte"
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "orderId": "ford_...",
    "status": "REJECTED",
    "refund": {
      "status": "PENDING",
      "amount": 8850
    }
  }
}
```

POST /api/v1/food/orders/{id}/ready

Marque prête pour retrait.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "orderId": "ford_...",
    "status": "READY_FOR_PICKUP",
    "readyAt": "2026-08-17T20:17:00Z"
  }
}
```

GET /api/v1/food/orders/{id}

Détail et suivi.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "id": "ford_...",
    "status": "ON_THE_WAY",
    "restaurant": {
      "id": "rest_...",
      "name": "Chez Awa"
    },
    "driver": {
      "id": "drv_...",
      "firstName": "Koffi",
      "phoneMasked": "+229*****89"
    },
    "etaMinutes": 9,
    "total": 8850,
    "currency": "XOF"
  }
}
```

Erreurs métier recommandées

Code	HTTP	Description
------	------	-------------

HBAExpress - Cahier des charges Backend

FOOD_ORDER_SERVICE_NOT_FOUND	404	Ressource principale introuvable.
VALIDATION_ERROR	400	Payload invalide ou champ obligatoire absent.
BUSINESS_RULE_VIOLATION	422	Règle métier non satisfaite.
CONFLICT	409	État incompatible, version concurrente ou idempotency conflict.
DEPENDENCY_UNAVAILABLE	503	Dépendance gRPC/Kafka/provider indisponible.

10.12 Payment Service

Abstraction des moyens de paiement, payment intents, webhooks, remboursements, réconciliation et provider routing.

Responsabilités et limites

Domaine : Commun. Base : PostgreSQL (payment_db) + Kafka + Redis idempotency; données carte jamais stockées. Le service est propriétaire de ses données et n'autorise aucun accès SQL direct depuis un autre service.

Structure des données

Table / agrégat	Champs principaux
payment_intents	id UUID, order_type, order_id, user_id, provider, method, amount, currency, status, provider_reference, idempotency_key
payment_transactions	id, payment_intent_id, type, amount, status, raw_reference, created_at
refunds	id, payment_intent_id, amount, reason, status, provider_reference

Contrats gRPC exposés

RPC	Usage
CreatePaymentIntent(orderRef,amount,method)->Intent	CreatePaymentIntent
GetPaymentStatus(intentId)	GetPaymentStatus
RefundPayment(intentId,amount,reason)	RefundPayment

Événements Kafka

Direction	Topics / événements
Publie	payment.created, payment.succeeded, payment.failed, payment.refunded
Consomme	marketplace.order.created, food.order.created

API REST/JSON exposée via BFF/Gateway

POST /api/v1/payments/intents

Crée une intention de paiement.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	201 Created
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "orderType": "FOOD",
  "orderId": "ford...",
  "method": "MTN_MOMO",
  "amount": 8850,
  "currency": "XOF",
  "payer": {
    "phoneNumber": "+2290197000000"
  }
}
```

```
}
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "paymentIntentId": "pay_...",
    "status": "REQUIRES_ACTION",
    "provider": "MTN_MOMO",
    "nextAction": {
      "type": "MOBILE_APPROVAL",
      "expiresAt": "2026-08-17T20:10:00Z"
    }
  }
}
```

GET /api/v1/payments/intents/{id}

Etat d'un paiement.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "id": "pay_...",
    "orderId": "ford_...",
    "amount": 8850,
    "currency": "XOF",
    "status": "SUCCEEDED",
    "provider": "MTN_MOMO",
    "paidAt": "2026-08-17T20:01:22Z"
  }
}
```

POST /api/v1/payments/{id}/refunds

Demande un remboursement.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	202 Accepted
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "amount": 8850,
  "reason": "RESTAURANT_REJECTED"
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "refundId": "ref_...",
    "status": "PENDING",
    "amount": 8850,
    "currency": "XOF"
  }
}
```

POST /api/v1/payments/webhooks/{provider}

Endpoint public signé pour webhooks fournisseur.

Propriété	Valeur
Authentification	Provider signature
Réponse nominale	202 Accepted
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "providerEventId": "evt_...",
  "status": "SUCCESSFUL",
  "reference": "provider-ref"
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "accepted": true
  }
}
```

Erreurs métier recommandées

Code	HTTP	Description
PAYMENT_SERVICE_NOT_FOUND	404	Ressource principale introuvable.
VALIDATION_ERROR	400	Payload invalide ou champ obligatoire absent.
BUSINESS_RULE_VIOLATION	422	Règle métier non satisfaite.
CONFLICT	409	État incompatible, version concurrente ou idempotency conflict.
DEPENDENCY_UNAVAILABLE	503	Dépendance gRPC/Kafka/provider indisponible.

10.13 Wallet & Settlement Service

Ledger financier, commissions plateforme, soldes marchands/livreurs, payouts et rapprochement.

Responsabilités et limites

Domaine : Commun. Base : PostgreSQL (wallet_db), modèle double-entry immutable. Le service est propriétaire de ses données et n'autorise aucun accès SQL direct depuis un autre service.

Structure des données

Table / agrégat	Champs principaux
accounts	id UUID, owner_type, owner_id, currency, status
ledger_entries	id UUID, transaction_id, account_id, direction DEBIT CREDIT, amount, balance_after, created_at
wallet_transactions	id UUID, type, reference_type, reference_id, amount, status
payouts	id, owner_id, amount, destination_type, destination_ref, status

Contrats gRPC exposés

RPC	Usage
GetBalance(ownerType,ownerId)	GetBalance
CreditSale(orderRef,split)	CreditSale
CreatePayout(ownerId,amount,destination)	CreatePayout

Événements Kafka

Direction	Topics / événements
Publie	wallet.credited, wallet.debited, payout.requested, payout.completed
Consomme	payment.succeeded, marketplace.order.delivered, food.order.delivered

API REST/JSON exposée via BFF/Gateway

GET /api/v1/wallets/me

Retourne le wallet professionnel/livreur.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "availableBalance": 250000,
    "pendingBalance": 75000,
    "currency": "XOF",
    "lastUpdatedAt": "2026-08-17T19:40:00Z"
  }
}
```

```

    }
  }

```

GET /api/v1/wallets/me/transactions

Liste le ledger métier.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```

{
  "success": true,
  "data": [
    {
      "id": "wtx_...",
      "type": "SALE_CREDIT",
      "amount": 15000,
      "currency": "XOF",
      "status": "COMPLETED",
      "reference": "ord_m_..."
    },
    {
      "id": "wtx_2",
      "type": "PLATFORM_COMMISSION",
      "amount": -1500,
      "currency": "XOF",
      "status": "COMPLETED"
    }
  ],
  "meta": {
    "page": 1,
    "pageSize": 20,
    "total": 91
  }
}

```

POST /api/v1/wallets/me/payouts

Demande un retrait.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	202 Accepted
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```

{
  "amount": 50000,
  "destination": {
    "type": "MTN_MOMO",
    "phoneNumber": "+2290197000000"
  }
}

```

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "payoutId": "po_...",
    "amount": 50000,
    "currency": "XOF",
    "status": "PENDING"
  }
}

```

```
}  
}
```

Erreurs métier recommandées

Code	HTTP	Description
WALLET_AND_SETTLEMENT_SERVICE_NOT_FOUND	404	Ressource principale introuvable.
VALIDATION_ERROR	400	Payload invalide ou champ obligatoire absent.
BUSINESS_RULE_VIOLATION	422	Règle métier non satisfaite.
CONFLICT	409	État incompatible, version concurrente ou idempotency conflict.
DEPENDENCY_UNAVAILABLE	503	Dépendance gRPC/Kafka/provider indisponible.

10.14 Delivery Service

Orchestration livraison commune Marketplace/Food : demande, dispatch, livreur, tracking, preuve de livraison et tarification.

Responsabilités et limites

Domaine : Delivery. Base : PostgreSQL (delivery_db) + Redis GEO pour disponibilité/positions + TimescaleDB optionnel pour traces GPS. Le service est propriétaire de ses données et n'autorise aucun accès SQL direct depuis un autre service.

Structure des données

Table / agrégat	Champs principaux
deliveries	id UUID, source_type FOOD MARKETPLACE, source_order_id, pickup_json, dropoff_json, priority, status, driver_id, fee, distance_m, eta_s
drivers	id UUID, user_id, status, vehicle_type, rating, current_lat, current_lng
driver_shifts	id, driver_id, started_at, ended_at, status
delivery_status_history	id, delivery_id, status, lat, lng, actor_id, created_at
proofs	id, delivery_id, type, object_url, otp_hash, signed_at

Contrats gRPC exposés

RPC	Usage
QuoteDelivery(pickup,dropoff,priority)	QuoteDelivery
CreateDelivery(orderRef)	CreateDelivery
GetDelivery(id)	GetDelivery
CancelDelivery(id)	CancelDelivery
GetDriver(driverId)	GetDriver

Événements Kafka

Direction	Topics / événements
Publie	delivery.created, delivery.assigned, delivery.picked_up, delivery.on_the_way, delivery.delivered, delivery.failed, driver.location.updated
Consomme	food.order.accepted, food.order.ready, marketplace.order.ready_for_pickup

API REST/JSON exposée via BFF/Gateway

POST /api/v1/deliveries/quote

Calcule frais/ETA.

Propriété	Valeur
Authentification	Bearer JWT

Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "sourceType": "FOOD",
  "pickup": {
    "lat": 6.37,
    "lng": 2.39
  },
  "dropoff": {
    "lat": 6.39,
    "lng": 2.42
  },
  "priority": "EXPRESS"
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "distanceMeters": 5200,
    "estimatedMinutes": 24,
    "fee": 1000,
    "currency": "XOF",
    "quoteId": "dq...",
    "expiresInSeconds": 300
  }
}
```

GET /api/v1/deliveries/{id}

Suivi de livraison.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "id": "del...",
    "sourceType": "FOOD",
    "sourceOrderId": "ford...",
    "status": "ON_THE_WAY",
    "driver": {
      "id": "drv...",
      "firstName": "Koffi",
      "rating": 4.9,
      "vehicle": "MOTORBIKE"
    },
    "tracking": {
      "lat": 6.381,
      "lng": 2.405,
      "updatedAt": "2026-08-17T20:11:00Z"
    },
    "etaMinutes": 9
  }
}
```

POST /api/v1/drivers/me/availability

Le livreur passe online/offline.

Propriété	Valeur
-----------	--------

Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "available": true,
  "vehicleType": "MOTORBIKE"
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "driverId": "drv_...",
    "availability": "ONLINE"
  }
}
```

POST /api/v1/deliveries/{id}/accept

Le livreur accepte une mission.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "deliveryId": "del_...",
    "driverId": "drv_...",
    "status": "ASSIGNED"
  }
}
```

POST /api/v1/deliveries/{id}/pickup

Confirme le retrait.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "pickupCode": "4821"
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "deliveryId": "del_...",
    "status": "PICKED_UP",
    "pickedUpAt": "2026-08-17T20:15:00Z"
  }
}
```

```

    }
  }

```

POST /api/v1/deliveries/{id}/complete

Preuve de livraison.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```

{
  "delivery0tp": "7312",
  "proofPhotoId": "media_..."
}

```

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "deliveryId": "del_...",
    "status": "DELIVERED",
    "deliveredAt": "2026-08-17T20:28:00Z"
  }
}

```

Erreurs métier recommandées

Code	HTTP	Description
DELIVERY_SERVICE_NOT_FOUND	404	Ressource principale introuvable.
VALIDATION_ERROR	400	Payload invalide ou champ obligatoire absent.
BUSINESS_RULE_VIOLATION	422	Règle métier non satisfaite.
CONFLICT	409	État incompatible, version concurrente ou idempotency conflict.
DEPENDENCY_UNAVAILABLE	503	Dépendance gRPC/Kafka/provider indisponible.

10.15 Notification Service

Notifications push, SMS, email et templates transactionnels avec préférences, retries et suivi de livraison.

Responsabilités et limites

Domaine : Commun. Base : PostgreSQL (notification_db) + Kafka; providers externes FCM/APNs, Twilio/agrégateur SMS, SendGrid/SES. Le service est propriétaire de ses données et n'autorise aucun accès SQL direct depuis un autre service.

Structure des données

Table / agrégat	Champs principaux
notification_templates	id, code, channel, locale, subject_template, body_template, version
notifications	id UUID, user_id, channel, template_code, payload_json, status, provider_message_id, sent_at
notification_preferences	user_id, event_code, push, sms, email

Contrats gRPC exposés

RPC	Usage
SendNotification(userId,template,payload)	SendNotification
GetPreferences(userId)	GetPreferences

Événements Kafka

Direction	Topics / événements
Publie	notification.sent, notification.failed
Consomme	payment.succeeded, food.order.accepted, delivery.assigned, delivery.delivered, marketplace.order.paid

API REST/JSON exposée via BFF/Gateway

GET /api/v1/notifications

Centre de notifications.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```
{
  "success": true,
  "data": [
    {
      "id": "not_...",
      "type": "ORDER",
      "title": "Commande acceptée",
      "message": "Chez Awa prépare votre commande.",
      "read": false,
      "createdAt": "2026-08-17T20:03:00Z"
    }
  ]
}
```

```

    },
    "meta": {
      "unreadCount": 3,
      "page": 1,
      "pageSize": 20
    }
  }
}

```

POST /api/v1/notifications/{id}/read

Marque comme lue.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "id": "not_...",
    "read": true
  }
}

```

PUT /api/v1/notifications/preferences

Met à jour les préférences.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```

{
  "orderUpdates": {
    "push": true,
    "sms": true,
    "email": false
  },
  "marketing": {
    "push": false,
    "sms": false,
    "email": false
  }
}

```

Exemple de réponse JSON

```

{
  "success": true,
  "data": {
    "updated": true
  }
}

```

Erreurs métier recommandées

Code	HTTP	Description
NOTIFICATION_SERVICE_NOT_FOUND	404	Ressource principale introuvable.

HBAExpress - Cahier des charges Backend

VALIDATION_ERROR	400	Payload invalide ou champ obligatoire absent.
BUSINESS_RULE_VIOLATION	422	Règle métier non satisfaite.
CONFLICT	409	État incompatible, version concurrente ou idempotency conflict.
DEPENDENCY_UNAVAILABLE	503	Dépendance gRPC/Kafka/provider indisponible.

10.16 Promotion Service

Coupons et promotions applicables au Marketplace/Food avec règles, budgets, usages et éligibilité.

Responsabilités et limites

Domaine : Commun. Base : PostgreSQL (promotion_db) + Redis pour compteurs rapides. Le service est propriétaire de ses données et n'autorise aucun accès SQL direct depuis un autre service.

Structure des données

Table / agrégat	Champs principaux
promotions	id UUID, scope FOOD MARKETPLACE GLOBAL, type PERCENT FIXED FREE_DELIVERY, value, starts_at, ends_at, budget, status
promotion_rules	id, promotion_id, rule_type, rule_json
coupons	id, promotion_id, code UNIQUE, max_uses, per_user_limit
coupon_usages	id, coupon_id, user_id, order_id, discount_amount, created_at

Contrats gRPC exposés

RPC	Usage
EvaluatePromotion(context)->Discount	EvaluatePromotion
ReserveCoupon(code,userId,cardId)	ReserveCoupon
CommitCoupon(reservationId,orderId)	CommitCoupon

Événements Kafka

Direction	Topics / événements
Publie	promotion.created, promotion.exhausted, coupon.used
Consomme	marketplace.order.cancelled, food.order.cancelled

API REST/JSON exposée via BFF/Gateway

POST /api/v1/promotions/validate

Valide un coupon pour un panier.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "code": "WELCOME10",
  "scope": "FOOD",
  "cardId": "fcart_..."
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "valid": true,
    "promotionId": "promo_...",
    "discount": 700,
    "currency": "XOF",
    "message": "10% de réduction"
  }
}
```

POST /api/v1/merchant/promotions

Crée une promotion marchand.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	201 Created
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de requête JSON

```
{
  "scope": "FOOD",
  "type": "PERCENT",
  "value": 15,
  "startsAt": "2026-08-20T00:00:00Z",
  "endsAt": "2026-08-31T23:59:59Z",
  "rules": {
    "minimumSubtotal": 5000
  }
}
```

Exemple de réponse JSON

```
{
  "success": true,
  "data": {
    "id": "promo_...",
    "status": "SCHEDULED"
  }
}
```

GET /api/v1/merchant/promotions

Liste des campagnes.

Propriété	Valeur
Authentification	Bearer JWT
Réponse nominale	200 OK
Idempotency-Key	Obligatoire pour POST de création/paiement/checkout; recommandé sinon

Exemple de réponse JSON

```
{
  "success": true,
  "data": [
    {
      "id": "promo_...",
      "type": "PERCENT",
      "value": 15,
      "status": "SCHEDULED",
      "uses": 0,
      "budgetRemaining": 100000
    }
  ]
}
```


Erreurs métier recommandées

Code	HTTP	Description
PROMOTION_SERVICE_NOT_FOUND	404	Ressource principale introuvable.
VALIDATION_ERROR	400	Payload invalide ou champ obligatoire absent.
BUSINESS_RULE_VIOLATION	422	Règle métier non satisfaite.
CONFLICT	409	État incompatible, version concurrente ou idempotency conflict.
DEPENDENCY_UNAVAILABLE	503	Dépendance gRPC/Kafka/provider indisponible.

11. Flux métier inter-services

11.1 Checkout Marketplace

Étape	Mode	Action
1	REST/JSON	Client -> Client BFF : POST /marketplace/orders/checkout
2	gRPC	BFF -> Marketplace Cart : ValidateCart
3	gRPC	Order -> Inventory : ReserveStock
4	gRPC	Order -> Promotion : Commit/Reserve discount
5	DB	Order créé PENDING_PAYMENT + Outbox
6	Kafka	marketplace.order.created
7	gRPC	Payment crée payment intent
8	Webhook/provider	Confirmation opérateur
9	Kafka	payment.succeeded
10	Consumer	Order passe PAID; Inventory commit réservation
11	Kafka	marketplace.order.paid -> Notification/Wallet
12	Kafka	ready_for_pickup -> Delivery Service
13	Kafka	delivery.delivered -> Order DELIVERED + settlement

11.2 Checkout Food

Étape	Mode	Action
1	REST/JSON	Client -> Client BFF : POST /food/orders/checkout
2	gRPC	Food Cart valide menu + restaurant ouvert
3	gRPC	Delivery quote pour frais/ETA
4	DB	Food Order PENDING_PAYMENT
5	gRPC/provider	Payment intent
6	Kafka	payment.succeeded -> FoodOrder PAID
7	Push/Kafka	Restaurant reçoit commande
8	REST seller BFF	Restaurant accepte + prep time
9	Kafka	food.order.accepted
10	Consumer	Delivery prépare dispatch
11	Kafka	food.order.ready -> assignation/pickup

12	Kafka	delivery.picked_up/on_the_way/delivered
13	Consumer	Food Order DELIVERED + Wallet settlement

11.3 Saga et compensations

Incident	Compensation
Paiement réussi mais restaurant refuse	FoodOrder REJECTED -> Payment Refund -> notification client
Stock réservé mais paiement échoue	Inventory ReleaseReservation
Livraison impossible après pickup	Delivery FAILED -> support workflow; remboursement partiel selon politique
Coupon réservé mais checkout abandonné	Expiration TTL + release coupon
Kafka redelivery	Consumer idempotent; processed_events unique(event_id, consumer_name)

12. Infrastructure & déploiement

Composant	Choix recommandé
Conteneurs	Docker
Orchestrateur	Kubernetes
Ingress/API Gateway	Kong / Traefik / NGINX selon standard équipe
Service discovery	Kubernetes DNS
Kafka	3 brokers minimum en production ou service managé
PostgreSQL	Instances/DB séparées logiquement; HA + PITR
Redis	Cluster/Sentinel selon charge
Object storage	S3 compatible / MinIO
Observabilité	OpenTelemetry + Prometheus + Grafana + Loki/ELK + Tempo/Jaeger
CI/CD	GitHub Actions / GitLab CI -> registry -> Helm/Argo CD
Secrets	Vault / External Secrets Operator

13. Structure recommandée d'un microservice

```

service-name/
├── src/
│   ├── Domain/           # Agrégats, entités, value objects, domain events
│   ├── Application/       # Use cases, commands, queries, DTO internes
│   ├── Infrastructure/    # EF/ORM, Kafka, Redis, providers, repositories
│   ├── Interfaces/
│   │   ├── Grpc/         # gRPC server/client
│   │   ├── Kafka/        # producers/consumers
│   │   └── Http/         # health/admin internes si nécessaire
│   └── Bootstrap/        # DI, config, startup
├── proto/                # .proto versionnés ou package contracts
├── tests/
│   ├── unit/
│   └── integration/

```

14. Résilience gRPC

Paramètre	Valeur/approche
Deadline	Définie sur chaque appel; 200-1500 ms selon criticité
Retry	Seulement opérations idempotentes; exponential backoff + jitter
Circuit breaker	Sur dépendances instables/providers
Bulkhead	Pools séparés pour fonctions critiques
Fallback	Read cache seulement si cohérence acceptable
Health	gRPC health checking + readiness/liveness
Load balancing	gRPC client-side via Kubernetes/service mesh selon stack

15. Observabilité et SLO

Indicateur	Cible initiale
API disponibilité	>= 99,9% hors maintenance
p95 lecture BFF	< 500 ms hors providers externes
p95 gRPC interne	< 150 ms pour lectures simples
Kafka consumer lag	Alerte si lag persistant > seuil 2-5 min
Taux erreur 5xx	< 1%
Traçabilité	100% requêtes avec requestId; sampling traces adapté
RPO PostgreSQL	<= 15 min cible initiale
RTO	<= 60 min cible initiale

16. Tests et qualité

Type	Exigence
Unitaires	Domain/Application, règles métier critiques
Intégration	DB réelle containerisée, Redis, Kafka via Testcontainers
Contract gRPC	Compatibilité Protobuf backward-compatible
Contract REST	OpenAPI + snapshot/consumer contract
E2E	Checkout Marketplace, Food, Payment, Delivery, refund
Performance	k6/Gatling sur BFF + gRPC + Kafka
Sécurité	SAST, dependency scan, container scan, DAST préprod

17. Definition of Done backend

- OpenAPI/BFF documenté et exemples JSON validés
- Proto gRPC versionné et contract tests verts
- Migration DB réversible ou stratégie forward-only documentée
- Événements Kafka documentés, schéma enregistré, producer/consumer idempotents
- Logs structurés sans secrets/PII sensible
- Metrics et traces OpenTelemetry présentes
- Timeout/retry/circuit breaker configurés
- Tests unitaires + intégration + E2E pertinents
- Docker image non-root et scan sécurité acceptable
- Runbook et alertes de production renseignés

18. Annexes - conventions techniques

Contexte propagé

```
{
  "requestId": "req_01J...",
  "traceId": "00-...",
  "actor": {
    "userId": "u_...",
    "roles": [
      "SELLER"
    ]
  },
  "idempotencyKey": "01J...",
  "locale": "fr-BJ"
}
```

Types canonisés

```
{
  "money": {
    "amount": 15000,
    "currency": "XOF"
  },
  "dateTime": "RFC3339 UTC",
  "id": "UUID v7",
  "phoneNumber": "E.164",
  "coordinates": {
    "lat": 6.3703,
    "lng": 2.3912
  }
}
```

Note : les exemples JSON sont des contrats externes illustratifs à stabiliser dans OpenAPI. Les échanges synchrones entre microservices doivent utiliser Protobuf/gRPC et non JSON. Les noms de tables sont indicatifs : le schéma final peut évoluer sans modifier le contrat public, tant que les invariants métier et la compatibilité des APIs sont respectés.

19. Standard des événements Kafka

Tous les événements métier HBAExpress utilisent une enveloppe commune. Cette convention s'applique aux domaines Marketplace, Food, Delivery et aux services communs. L'objectif est d'obtenir des messages traçables, versionnés, idempotents et compatibles avec une évolution indépendante des microservices.

19.1 Enveloppe canonique

```
{
  "eventId": "0198a43c-7a40-7c31-9d9f-2fa2d76e0c51",
  "eventType": "marketplace.order.created",
  "eventVersion": 1,
  "occurredAt": "2026-08-17T19:05:31.442Z",
  "producer": "marketplace-order-service",
  "environment": "production",
  "correlationId": "req_01K2Y7H8B...",
  "causationId": "cmd_01K2Y7FQ...",
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736",
  "aggregate": {
    "type": "MarketplaceOrder",
    "id": "0198a43b-d97a-7bb7-b18c-9c7a2e8a0e50",
    "version": 4
  },
  "actor": {
    "type": "CUSTOMER",
    "id": "0198a410-b53a-7f30-915d-4fd8f5272e87"
  },
  "tenantId": "hba-bj",
  "partitionKey": "0198a43b-d97a-7bb7-b18c-9c7a2e8a0e50",
  "data": {
    "orderId": "0198a43b-d97a-7bb7-b18c-9c7a2e8a0e50",
    "customerId": "0198a410-b53a-7f30-915d-4fd8f5272e87",
    "sellerId": "0198a3ef-2b1a-704e-b89c-9f276b8df211",
    "total": { "amount": 18500, "currency": "XOF" },
    "status": "PENDING_PAYMENT"
  },
  "metadata": {
    "schema": "hba.marketplace.order.created.v1",
    "contentType": "application/json",
    "locale": "fr-BJ"
  }
}
```

Champ	Obligatoire	Rôle
eventId	Oui	UUID v7 unique du message. Clé principale d'idempotence côté consumer.
eventType	Oui	Nom métier stable : <domaine>.<agrégat>.<action>.
eventVersion	Oui	Version entière du contrat de l'événement.
occurredAt	Oui	Date UTC RFC3339 de production de l'événement métier.
producer	Oui	Nom du microservice producteur.
environment	Oui	local, staging ou production.
correlationId	Oui	Identifiant commun à tout un flux métier distribué.
causationId	Recommandé	Commande ou événement ayant causé cet événement.
traceId	Oui	Trace OpenTelemetry propagée entre gRPC et Kafka.
aggregate	Oui	Type, ID et version de l'agrégat métier concerné.
actor	Selon cas	Utilisateur/service à l'origine de l'action.
tenantId	Oui	Périmètre logique/market de la donnée.
partitionKey	Oui	Clé utilisée pour préserver l'ordre d'un agrégat.
data	Oui	Payload métier spécifique à eventType/eventVersion.
metadata	Oui	Schéma, type de contenu, locale et extensions techniques.

19.2 Règles de nommage, topics et partitionnement

Convention eventType : <domain>.<aggregate>.<past-action>. Exemples : food.order.accepted, payment.succeeded, delivery.driver.assigned.

Convention topic : hba.<env>.<domain>.<aggregate>.v<major>. Exemple : hba.prod.food.order.v1.

Partition key : utiliser l'identifiant de l'agrégat principal (orderId, paymentId, deliveryId). Tous les événements d'une même commande sont ainsi routés vers la même partition et conservent leur ordre Kafka.

Exemple de headers Kafka

```
{
  "event-id": "0198a43c-7a40-7c31-9d9f-2fa2d76e0c51",
  "event-type": "food.order.accepted",
  "event-version": "1",
  "correlation-id": "req_01K2Y7H8B...",
  "traceparent": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01",
  "content-type": "application/json"
}
```

19.3 Exemple : FoodOrderAccepted

```
{
  "eventId": "0198a53c-73ae-79ad-b660-8cf30c56d109",
  "eventType": "food.order.accepted",
  "eventVersion": 1,
  "occurredAt": "2026-08-17T19:08:10.001Z",
  "producer": "food-order-service",
  "environment": "production",
  "correlationId": "req_01K2Y7H8B...",
  "causationId": "0198a52b-8810-7cb7-86cf-a1e152ea7465",
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736",
  "aggregate": {
    "type": "FoodOrder",
    "id": "0198a520-c2d4-7512-95f5-70cb8f68e98d",
    "version": 7
  },
  "actor": {
    "type": "RESTAURANT",
    "id": "0198a211-78d1-72bb-82fd-1426e6151ef1"
  },
  "tenantId": "hba-bj",
  "partitionKey": "0198a520-c2d4-7512-95f5-70cb8f68e98d",
  "data": {
    "orderId": "0198a520-c2d4-7512-95f5-70cb8f68e98d",
    "restaurantId": "0198a211-78d1-72bb-82fd-1426e6151ef1",
    "acceptedAt": "2026-08-17T19:08:10.001Z",
    "estimatedPreparationMinutes": 25,
    "readyAt": "2026-08-17T19:33:10.001Z",
    "deliveryRequired": true
  },
  "metadata": {
    "schema": "hba.food.order.accepted.v1",
    "contentType": "application/json",
    "locale": "fr-BJ"
  }
}
```

Consumers typiques : Delivery Service crée une demande de livraison ; Notification Service informe le client ; Analytics projette l'événement dans ses vues de reporting. Aucun consumer ne doit modifier directement la base du Food Order Service.

19.4 Exemple : PaymentSucceeded

```
{
  "eventId": "0198a61f-599a-7c32-9837-a7edebf805d1",
  "eventType": "payment.succeeded",
  "eventVersion": 1,
  "occurredAt": "2026-08-17T19:10:44.901Z",
  "producer": "payment-service",
  "environment": "production",
  "correlationId": "req_01K2Y7H8B...",
  "causationId": "0198a60d-bf5c-79b5-9c08-91728bcaef83",
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736",
  "aggregate": {
    "type": "Payment",
    "id": "0198a60f-1e34-79de-878d-f24c5df66e72",
    "version": 3
  },
  "actor": { "type": "SYSTEM", "id": "payment-service" },
  "tenantId": "hba-bj",
  "partitionKey": "0198a60f-1e34-79de-878d-f24c5df66e72",
  "data": {
    "paymentId": "0198a60f-1e34-79de-878d-f24c5df66e72",
    "orderId": "0198a520-c2d4-7512-95f5-70cb8f68e98d",
    "orderType": "FOOD",
    "provider": "MTN_MOMO",
    "providerTransactionId": "12605134700",
    "amount": { "amount": 18500, "currency": "XOF" },
    "paidAt": "2026-08-17T19:10:44.100Z"
  },
  "metadata": {
    "schema": "hba.payment.succeeded.v1",
    "contentType": "application/json",
    "locale": "fr-BJ"
  }
}
```

19.5 Idempotence et traitement consumer

Chaque consumer conserve les eventId déjà traités dans une table/inbox locale. La transaction métier et l'enregistrement de l'événement consommé doivent être atomiques lorsque le stockage le permet.

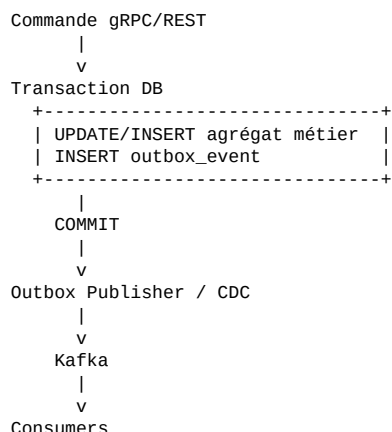
1. Kafka délivre le message
2. Valider eventType + eventVersion + schéma
3. Vérifier inbox(eventId)
4. Si déjà traité -> ACK sans rejouer le métier
5. Sinon -> exécuter la transaction métier
6. Insérer eventId dans consumer_inbox
7. Commit
8. ACK Kafka

Structure recommandée de consumer_inbox

Colonne	Type	Description
event_id	UUID / PK	Identifiant Kafka logique déjà traité
consumer_name	VARCHAR(120)	Nom du consumer / handler
event_type	VARCHAR(160)	Type d'événement
processed_at	TIMESTAMPTZ	Date de succès
correlation_id	VARCHAR(100)	Corrélation distribuée

19.6 Publication fiable : Transactional Outbox

Un service ne doit pas faire un commit SQL puis publier directement dans Kafka comme deux opérations indépendantes. L'état métier et le message à publier sont écrits dans la même transaction locale. Un publisher relaie ensuite l'Outbox vers Kafka.



Colonne	Type	Description
id	UUID v7 / PK	Identifiant de l'enregistrement Outbox
event_id	UUID v7 / UNIQUE	eventId de l'enveloppe Kafka
aggregate_type	VARCHAR(100)	Type d'agrégat
aggregate_id	UUID	Identifiant d'agrégat / partition key
event_type	VARCHAR(160)	Type de l'événement
event_version	INT	Version du contrat
payload	JSONB	Enveloppe complète
created_at	TIMESTAMPTZ	Création locale
published_at	TIMESTAMPTZ NULL	Publication Kafka réussie
attempts	INT	Nombre de tentatives

19.7 Retry, DLQ et compatibilité

Retry : les erreurs transitoires utilisent un backoff exponentiel avec jitter. **DLQ** : après le nombre maximal de tentatives, le message est envoyé vers `hba.<env>.<domain>.<aggregate>.v1.dlq` avec la cause d'échec. Les erreurs métier définitives ne doivent pas être rejouées indéfiniment.

Évolution : ajouter des champs optionnels est compatible dans une même version majeure. Renommer/supprimer un champ ou modifier sa sémantique impose une nouvelle version majeure du schéma/topic. Un consumer doit ignorer les champs inconnus et déclarer explicitement les versions qu'il supporte.

Rétention : la durée dépend du domaine. Les événements d'intégration doivent rester assez longtemps pour permettre replay, reprise après incident et reconstruction de projections. Les données personnelles inutiles ne doivent jamais être placées dans les événements.

19.8 Checklist obligatoire pour chaque nouvel événement

- eventType et propriétaire métier définis
- schéma versionné et enregistré
- topic, partition key et rétention définis
- producer et consumers documentés
- payload minimal : pas de secret, token ou donnée personnelle inutile
- Outbox côté producer
- Inbox/idempotence côté consumer

- correlationId + traceId propagés
- retry + DLQ + métriques configurés
- contract tests producer/consumer ajoutés
- stratégie de compatibilité et de dépréciation documentée